DSA03-NATURAL LANGUAGE PROCESSING
ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: K DEEPTHI

REG NO: 192472211
Duration: 1 Week
Total Marks: 50
Submission Date: ___03/11/2026_____

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.
- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.

- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

ANSWER

Levels of linguistic analysis

Level	Meaning	Hospital-record example / relevance
Phonology	Sound patterns of a language.	Not central for typed English text; becomes relevant indirectly when speech or OCR spelling errors are considered.
Morphology	Structure of words and affixes.	Essential: fracture/fractures; operate/operated; nephrectomy/nephrectomies.
Syntax	Arrangement of words in sentences.	Useful for interpreting phrases such as “no evidence of infection”; a basic search layer uses limited syntax.
Semantics	Meaning of words and sentences.	Important for distinguishing related but non-equivalent terms, such as renal and

		nephrectomy.
Pragmatics	Meaning in context and speaker intention.	Limited role in initial normalisation; may help interpret clinical shorthand and negation.
Discourse	Relations across sentences or a whole document.	Useful for linking a diagnosis with later references, but not required for basic token normalisation.

Model families

State machines: best for validating structured formats such as dates, identifiers and codes because they accept or reject strings through explicit transitions.

Formal rule systems: best for predictable token and spelling transformations, such as lowercasing, abbreviation expansion and inflectional suffix rules.

Probabilistic models: best for uncertain OCR correction, language identification and context-sensitive ambiguity. They require representative data and are less transparent than exact rules.

A state machine would be unnecessarily rigid for deciding the intended meaning of an ambiguous abbreviation. A probabilistic model is unnecessary for a simple fixed date format. A hand-written rule is unsuitable for every noisy OCR variation because the number of possible errors is large.

Ambiguity and search

Lexical ambiguity: “cold” may refer to a symptom or a temperature description.

Morphological ambiguity: “findings” may be a plural noun or part of a verb phrase.

Segmentation ambiguity: “500mg” may be one quantity token or two tokens, “500” and “mg”.

Abbreviation ambiguity: “MS” may have more than one clinical expansion depending on context.

Automata use state-space traversal; regular-expression engines may use backtracking depending on implementation and pattern. Resources include processing time, memory, annotated examples and expert effort for creating and maintaining rules.

Proposed pipeline

Input records → regex entity extraction → maximal-munch tokeniser → case/date/unit normaliser → DFA validators → morphological FST → Porter stemmer comparison → searchable index

The input is de-identified clinical text. The output is a normalised token stream, extracted structured entities and one or more morphological analyses for each recognised word.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

ANSWER

Basic operators

Operator	Meaning	Example
Concatenation	Places expressions one after another.	[0-9][0-9] for two digits.
Disjunction	Matches either alternative.	mg|mcg
Kleene star *	Zero or more repetitions.	[0-9]*
Plus +	One or more repetitions.	[0-9]+
Optional ?	Zero or one occurrence.	-? for optional minus sign.
Character class	One character from a set.	[A-Z]
Range	A continuous character range.	[0-9]
Negated class	Any character not listed.	[^\s]+
Anchors	Start and end of input.	^ID[0-9]+$
Counters	A specified repetition count.	[0-9]{4}

Twelve entity patterns

The following patterns are starter patterns. They must be tested against the hospital's actual data and adjusted to local formats. They are examples of recognition rules, not proof that every valid clinical format has been covered.

Entity	Example regular expression	Matches / does not match
Patient ID	^PT[0-9]{6}$	PT123456; rejects PT123 and PT1234567.
Date (DD-MM-YYYY)	^(0[1-9]|[12][0-9]|3[01])[-/](0[1-9]|1[0-2])[-/][0-9]{4}$	18-09-2026; rejects 39-19-2026 (calendar validation still needed).
Drug + strength	^[A-Za-z][A-Za-z-]*[ ]*[0-9]+(\\.[0-9]+)?[ ]*(mg|mcg|g|ml)$	paracetamol 500mg; rejects a bare drug name.

Dosage frequency	^(OD|BD|TDS|QID|once daily|twice daily)$	BD; rejects unrelated words.
Lab value + unit	^[0-9]+(\\.[0-9]+)?[ ]*(mg/dL|g/dL|mmol/L|U/L)$	5.6 mmol/L; rejects missing unit.
Blood pressure	^[0-9]{2,3}/[0-9]{2,3}$	120/80; rejects 120-80.
Temperature	^[0-9]{2}(\\.[0-9])?[ ]?°?[CF]$	37.0C; rejects alphabetic text.
ICD code	^[A-Z][0-9][0-9](\\.[A-Z0-9]{1,4})?$	S72.0; rejects an unstructured word.
Ward and bed	^W[0-9]{1,2}-B[0-9]{1,3}$	W2-B14; rejects W-B.
Contact number	^[+]?[0-9]{10,13}$	9876543210; rejects short numbers.
Age	^[0-9]{1,3}[ ]?(years?|yrs?)$	45 years; rejects text without a number.
Symptom duration	^[0-9]+[ ]?(days?|weeks?|months?|years?)$	3 days; rejects “several days” unless separately handled.

Greedy and non-greedy matching

Text: "Give 500 mg daily; repeat 250 mg if required."

Greedy example: .*mg may extend from the first number to the last "mg".

Non-greedy example: .*?mg stops at the first possible "mg".

A greedy quantifier takes the longest span that still permits the whole expression to match. A non-greedy quantifier takes the shortest such span. For extracting individual dosage expressions, a bounded pattern such as [0-9]+\s?mg is safer than matching arbitrary text around “mg”.

Precedence and evaluation

Concatenation binds more tightly than disjunction. For example, ^A|B$ means (^A) or (B$), not ^(A|B)$. The grouped form ^(A|B)$ requires the entire input to be either A or B. For each

pattern, evaluation should record true positives, false positives and false negatives against manually labelled records. The source assignment does not provide a corpus or measured counts, so those results must be collected from the student's dataset rather than invented.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

ANSWER

Token definition

A token is a meaningful unit retained for search. The tokenizer should keep decimal values together, preserve common abbreviations, retain hyphenated clinical terms when they form one

expression, and keep a number with its unit when separation would lose useful context.

([A-Za-z]+(?:[.-][A-Za-z]+)*|[0-9]+(?:\.[0-9]+)?(?:\s?(?:mg|mcg|g|ml|mmol/L|mg/dL))?|[0-9]{1,2}[-/][0-9]{1,2}[-/][0-9]{2,4}|[^\w\s])

This is a practical starting expression. A production tokenizer should use ordered alternatives and test overlaps carefully. For example, “Dr.” should be preserved as an abbreviation rather than split into “Dr” and a full stop.

Maximal-munch examples

Input	Whitespace split	Maximal-munch tokens
BP 120/80 recorded	BP | 120/80 | recorded	BP | 120/80 | recorded
Take 500mg BD	Take | 500mg | BD	Take | 500mg | BD
Follow-up after 3.5 days	Follow-up | after | 3.5 | days	Follow-up | after | 3.5 | days

Whitespace splitting may keep some examples together by accident, but it does not define how punctuation, decimal values, abbreviations or attached units should be handled. Maximal-munch chooses the longest valid token at each position according to the token rules.

Normalisation rules

Case folding: convert ordinary alphabetic tokens to lowercase while preserving original text in a separate field.

Abbreviation expansion: map approved forms such as “BP” to “blood pressure”, “HR” to “heart rate” and “OD” to a locally verified dosage meaning. Ambiguous abbreviations must be context checked.

Date canonicalisation: convert recognised dates to YYYY-MM-DD only after validating the calendar date and knowing the source order.

Unit canonicalisation: map equivalent spellings to a standard unit label without changing the numeric value.

Before: "BP 120/80; Rx: Paracetamol 500 MG BD. Review on 18/09/2026."

After: "blood pressure 120/80; prescription: paracetamol 500 mg twice daily. review on 2026-09-18."

The example is illustrative; the abbreviation expansion must follow the hospital's approved

terminology.

Irreversible decisions and limits

Lowercasing loses original capitalisation, which may distinguish abbreviations from ordinary words.

Removing punctuation may destroy sentence boundaries or decimal meaning.

Expanding an ambiguous abbreviation can replace the original intended meaning with a wrong one.

The original token and source span should therefore be retained alongside the normalised value. A classical regular expression cannot recognise arbitrary balanced nested parentheses because that language is not regular; a stack-based parser or a context-free grammar is needed.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks
Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple (Q, Σ, δ, q_0, F), then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two way equivalence with regular expressions, and a valid non-regularity proof.

ANSWER

Formal definition

A finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$: Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states. A deterministic finite automaton (DFA) has exactly one next state for each state and input symbol.

Example DFA: patient identifier

For the format PT followed by six digits, use states q0 (start), q1 (after P), q2 (after PT), q3–q8 (after one through six digits), and a dead state for invalid input. q8 is accepting only when the input ends.

Current state	Input	Next state
q0	P	q1
q1	T	q2
q2	digit	q3
q3	digit	q4
q4	digit	q5
q5	digit	q6
q6	digit	q7
q7	digit	q8
q8	any extra character	dead

The same design method can validate a fixed-length contact number, a simple blood-pressure format and a ward-bed identifier. Each validator should have a transition diagram and a complete table including invalid transitions.

NFA with optional part

Consider an identifier that may contain an optional hyphen: PT123456 or PT-123456. An NFA can branch after PT: one path consumes a hyphen and the other proceeds directly to the first digit. An epsilon transition represents a move that consumes no input.

For PT-123456, the active state set after reading PT includes the branch that expects a hyphen and the branch that skips it. After reading the hyphen, only the digit path remains. PT123456 is accepted through the skip branch; PT--123456 is rejected because no transition accepts the second hyphen.

Subset construction and minimisation

Subset construction converts each NFA state-set into one DFA state. Begin with the epsilon-closure of the NFA start state. For each input symbol, move from every state in the set, then take the epsilon-closure of the result. Repeat until no new sets appear. DFA minimisation then merges states that cannot be distinguished by any remaining input suffix. The exact state counts depend on the detailed NFA transition table and should be calculated from the student's final design.

Thompson construction and pumping lemma

Thompson's construction creates an NFA from a regular expression using small fragments for symbols, concatenation, alternation and repetition. Conversely, state elimination can derive a regular expression from an automaton. Both directions describe the same regular language.

The pumping lemma states that for every regular language there is a pumping length p such that every string s in the language with length at least p can be split as $s = xyz$, where $|xy| \leq p$, $|y| > 0$, and xy^iz remains in the language for every $i \geq 0$. The language of balanced parentheses is not regular: pumping a segment within the first p opening brackets changes the number of opening brackets without changing the matching closing brackets. This connects to the limitation described in Task 3.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

ANSWER

Key terms

A morpheme is the smallest unit that carries meaning or grammatical function. A stem is the form to which an affix is attached. An affix is a bound morpheme added to a stem. Inflection changes grammatical form without normally changing the word class or core dictionary meaning. A paradigm is the set of related forms of a lexeme.

Inflection generally preserves category and is productive. Derivation can create a new lexeme

or change word category. Inflection usually occurs outside derivational material in English word formation.

Clinical inflectional paradigms

Lexeme	Illustrative forms	Feature
fracture	fracture, fractures	noun singular / plural
patient	patient, patients	noun singular / plural
operate	operate, operates, operated, operating	verb agreement / tense / aspect
treat	treat, treats, treated, treating	verb agreement / tense / aspect
severe	severe, severer, severest	adjective comparison

English spelling rules

Final silent e deletion: remove e before a vowel-initial suffix where required, as in operate + ing → operating. It does not apply to every word ending in e.

Consonant doubling: double a final consonant in stressed short-vowel patterns, as in admit → admitted. It must not be applied blindly to words with a different stress or spelling pattern.

Y-to-i: change consonant + y to i before many suffixes, as in injury → injuries; retain y before -ing, as in studying.

Sibilant plural: add -es after s, x, z, ch or sh, as in diagnosis → diagnoses. Exceptions and irregular forms require lexical entries.

Irregular forms and measurement

Irregular forms such as children, men, women, feet and went cannot be generated reliably by the regular suffix rules and should be stored in a separate exception lexicon. Clinical data may also contain irregular plurals such as diagnoses. Type-to-lemma ratio is calculated as the number of distinct surface word types divided by the number of distinct lemmas. To measure retrieval benefit, compare the query failures before and after morphological normalisation.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

ANSWER

Definition and affix inventory

Derivation forms a new lexeme by adding a prefix or suffix. It may change the word category or create a specialised meaning. The following examples are common in medical vocabulary; the corpus should be checked to confirm which are actually attested.

Affix	Example	Typical contribution
-itis	arthritis	inflammation; often forms a noun
-ectomy	nephrectomy	surgical removal; noun
-ology	cardiology	field of study; noun
-osis	stenosis	condition or process; noun
-aemia	anaemia	blood-related condition; noun
-ation	ventilation	process or result; noun
-able	treatable	capable of being treated; adjective
-ive	obstructive	having a stated property; adjective
nephro-	nephrology	kidney-related; prefix
hyper-	hypertension	high / excessive; prefix
hypo-	hypoglycaemia	low / deficient; prefix
re-	reoperation	again; prefix
un-	unhealed	not / opposite; prefix
-al	renal	relating to; adjective
-ic	cardiac	relating to; adjective

Derivational families and order

nephr-: renal / nephrology / nephrectomy. The root combines with different suffixes to create related but distinct clinical terms.

cardi-: cardiac / cardiology / cardiomegaly.

infect-: infect / infection / infectious / reinfection.

operat-: operate / operation / operative / reoperation.

treat-: treat / treatment / treatable / untreated.

obstruct-: obstruct / obstruction / obstructive.

Affix order is constrained. For example, re- attaches to a complete base in “reoperation”; a suffix that creates a noun may follow a verbal base in “ventilate → ventilation”. Arbitrary orders such as *-ectomy + nephro- attached in the reverse order are not valid English formations.

Non-compositional meaning and productivity

A derived term may have a specialised meaning that is not fully predictable from its parts. “Operation” in a hospital record may refer to a surgical procedure, not merely the general act of operating. Aggressive stemming can therefore merge words that are formally related but clinically distinct.

Productivity should be measured by counting distinct word types and total tokens containing each affix. An affix appearing with many new bases is more productive; an affix restricted to a small number of fixed technical words may be fossilised.

Search conflation decision

Conflating “fracture” and “fractures” helps retrieve plural mentions of the same condition. Connecting “infect” and “infection” may also help broaden a clinician's search. However, merging “renal” with “nephrectomy” can be misleading because one is a relation to the kidney and the other is a surgical procedure. Similarly, “hypertension” and “hypotension” must remain distinct. Derivational relatives should therefore be linked selectively rather than collapsed into one universal stem.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

ANSWER

Finite-state transducer

A finite-state transducer (FST) is a finite-state machine with two tapes. One tape represents lexical input (the stem and grammatical features); the other represents the surface word. A transition can consume a lexical symbol and emit a surface symbol. The same transducer can be used to analyse a surface form or generate a surface form from a lexical representation.

Lexicon and continuation classes

A small demonstration lexicon can be divided into continuation classes: noun stems, verb stems, adjective stems and affix classes. Each class specifies which suffixes may follow a stem. A full assignment lexicon should contain at least 300 stems, 20 inflectional affixes and 15 derivational affixes, as required by the source brief.

Lexical representation	Surface form	Analysis
fracture + N + PL	fractures	fracture + noun + plural
operate + V + PAST	operated	operate + verb + past
operate + V + PROG	operating	operate + verb + progressive
patient + N + PL	patients	patient + noun + plural
treat + V + PAST	treated	treat + verb + past

Parsing and generation

For “fractures”, the analyser starts from the surface side, follows transitions that consume f-r-a-c-t-u-r-e-s and outputs the lexical stem “fracture” plus the features +N +PL. In generation, the same path is traversed in the opposite direction: fracture +N +PL produces “fractures”. This demonstrates analysis and generation using one relation.

Ambiguity

A surface form may have multiple analyses. “Treating” can be a verb form in “the team is treating the patient” or a noun-like gerund in “treating the wound is important”. “Findings” may be a plural noun or a form used in a verbal construction. The FST should return all analyses licensed by its lexicon and morphology. A later context-sensitive component can rank them using surrounding words and sentence structure.

The complete submission should include at least five hand-traced forms, state sequences, symbols consumed and emitted, and the final lexical analyses. The examples above illustrate the expected format; the complete transition inventory must be built from the student's lexicon.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c __ d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

ANSWER

Why a lexicon transducer is not enough

A lexicon FST that simply concatenates a stem and an affix may produce incorrect spellings. Examples include study + s → *studys instead of studies; admit + ed → *admited instead of admitted; and operate + ing → *operateing instead of operating. Orthographic rules are needed to map lexical combinations to valid surface forms.

Rewrite rules

Rule	Illustrative rewrite	Context / example
Final e deletion	e → 0 / __ +V	operate + ing → operating
Consonant doubling	C → CC / Ý __ +V	admit + ed → admitted
Y-to-i	y → i / C __ + suffix	injury + s → injuries
Sibilant plural	+s → +es / sibilant __	diagnosis + s → diagnoses
Plural allomorph	+s → +es / ch, sh, x, z __	branch + s → branches

Here, 0 denotes deletion, C a consonant, V a vowel and +V a vowel-initial suffix. The precise context must be formalised and tested; simplified rules can over-generate.

Cascade and two-level morphology

Lexicon FST → morphotactic analysis → orthographic rule transducer 1 → rule transducer 2 → surface form

In a sequential cascade, one rule transducer feeds the next, so rule order may affect the result. A two-level morphology system instead relates lexical and surface symbols in parallel through constraints. A cascade is often easier to understand and debug step by step; a two-level model can express interacting spelling constraints without requiring a long sequence of intermediate rewrites.

Testing

Test at least twenty forms that exercise every rule, including regular and exception cases. Record expected surface form, generated form, pass/fail and explanation. Every failure should be classified as a missing lexical entry, incorrect continuation class, incomplete spelling rule or irregular exception. Actual success counts require running the implementation.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in

precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

ANSWER

Algorithm concepts

The Porter stemmer is a rule-based suffix-stripping algorithm for English. It does not require a dictionary. It uses consonant/vowel patterns and a measure m, which counts vowel-consonant sequences in the stem. The conditions *v, *d and *o test whether a stem contains a vowel, ends in a double consonant, or ends in a consonant-vowel-consonant pattern with specified restrictions.

Five-step outline

Step 1: remove common plurals and selected -ed / -ing endings, applying conditions and possible adjustments such as e insertion or consonant handling.

Step 2: replace selected longer suffixes when the stem measure is greater than zero.

Step 3: remove or replace another group of derivational suffixes when the measure condition is met.

Step 4: remove a further group of suffixes when the stem measure is greater than one.

Step 5: remove a final e or reduce a double l under the algorithm's conditions.

Exact rule ordering matters. A stemmer should be run using a tested implementation rather than a loosely approximated set of suffix deletions.

Example comparison

Word	Possible Porter stem	Morphological lemma	Comment
fractures	fractur	fracture	Stemmer removes final e; lemmatiser preserves dictionary lemma.
fractured	fractur	fracture	Both connect inflected form to base, but output differs.
patients	patient	patient	Plural suffix removed.
operating	oper	operate	Stemmer may shorten more than the lemma.
infection	infect	infection / infect	Derivational relation requires a policy; not simply an inflectional lemma.
diagnoses	diagnos	diagnosis	Irregular plural spelling needs lexical analysis.

Over-stemming occurs when unrelated words are conflated; under-stemming occurs when related forms remain separate. Both must be measured against manually annotated forms. The source assignment requires 200 annotated forms and 40 queries tested under no stemming, Porter stemming and full morphological analysis. Precision = relevant retrieved / total retrieved; recall = relevant retrieved / total relevant; F1 = 2 × precision × recall / (precision + recall).

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

ANSWER

Implementation plan

The brief permits Python's standard library and NLTK. The regular-expression module can implement tokenisation and entity patterns; explicit state-transition dictionaries can implement small DFAs; a set-of-states simulation can demonstrate NFA subset construction; dictionaries can hold stems and affixes; and NLTK can provide the Porter stemmer.

```
import re
from nltk.stem import PorterStemmer

text = "BP 120/80; Paracetamol 500mg BD."
tokens = re.findall(
    r"[A-Za-z]+(?:[.-][A-Za-z]+)*|[0-9]+(?:\.[0-9]+)?(?:mg|mcg|ml)?|[^\w\s]",
    text
)
normalised = [t.lower() for t in tokens]
stemmer = PorterStemmer()
stems = [stemmer.stem(t) for t in normalised if t.isalpha()]
print(tokens)
print(stems)
```

Testing strategy

Create de-identified or synthetic records covering discharge summaries, prescriptions, operation notes and nursing observations.

Run every entity pattern on positive and negative examples, including malformed dates and identifiers.

Test each DFA with accepted and rejected strings; compare NFA and subset-converted DFA outputs.

Test all regular and irregular morphological forms, including ambiguous examples.

Compare implementation output with the hand traces from Tasks 4, 7 and 9.

Reporting results

Pipeline stage	Items to report	Measurement
Tokenisation	Input records and output tokens	Tokens per second; rejected or unclassified tokens
Entity extraction	Candidate entities by type	TP, FP, FN, precision and recall
Automata	Accepted and rejected test strings	Accuracy on labelled test cases
Morphological analysis	Surface forms analysed	Coverage and ambiguity count
Porter stemmer	Alphabetic tokens stemmed	Runtime and over/under-stemming errors

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

ANSWER

Comparison

Approach	Advantages	Limitations
No normalisation	Fast, simple and preserves original wording.	Misses inflectional variants and format differences.
Porter stemming	Fast; no lexicon required; improves matching of many suffix variants.	May over-stem or under-stem; output may not be a real word or clinical lemma.
Finite-state morphology	Provides interpretable stems and grammatical features; supports generation and analysis.	Requires lexicon, rules, annotation and ongoing maintenance.

Recommendation

A hybrid design is recommended. Use regular expressions and deterministic automata for well-defined structured entities, a finite-state analyser for common clinical morphology, and Porter stemming as a fallback for ordinary English words that are not in the lexicon. Preserve the original text and attach confidence or provenance information to every normalised value.

This design balances precision and coverage. It avoids treating every derivational relative as identical and allows clinicians to inspect the original wording. Its failures include unseen abbreviations, misspellings, ambiguous terms, non-English morphology and incorrect local assumptions about date or dosage formats.

Effort and maintenance

Manual work includes collecting and de-identifying records, writing patterns, constructing the

stem and affix lexicon, annotating forms, reviewing ambiguous terms and testing changes. A statistical approach may become more economical when new vocabulary and noisy input grow faster than experts can maintain rules, provided enough representative labelled data and governance are available.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

ANSWER

A. IMPACT ANALYSIS (2 MARKS)

The new input contains OCR errors and Romanised regional-language text mixed with English. Examples include 0/O, 1/l and rn/m confusions, words split across line endings, lost diacritics and code-mixed phrases such as “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of tokens may be out of lexicon, misspelt or non-English.

A.1 Assumptions that no longer hold

- Clean typed input: OCR introduces character-level substitutions, insertions, deletions and line-break artefacts.
- Closed English vocabulary: regional-language words, names, abbreviations and misspellings may not exist in the lexicon.
- Exact matching: a strict DFA rejects near-matches even when the intended value is obvious to a human.
- English-only morphology: regional-language suffixes and code-mixed word order are not represented by the English analyser.

These changes are expected to reduce exact-pattern precision and recall and lower morphological coverage. The magnitude must be measured on a labelled noisy sample; it cannot be derived from the assignment statement alone.

B. EVALUATION OF THE ORIGINAL DESIGN (2 MARKS)

Three test records should be prepared: (1) OCR-only, with character confusions and broken words; (2) code-mixed, with regional-language tokens and English medical terms; and (3) a record containing both types of noise. Each record should be passed through the original pipeline without correction.

Input issue	**Likely first failure**	**Reason**
O/0 or l/1 confusion	Entity pattern / DFA	Exact character sequence no longer matches.
Word broken at line end	Tokenizer	One lexical item may become two tokens.
Unknown regional word	Morphological analyser	No matching stem or affix path exists.
Code-mixed phrase	English-only rules	Token may be recognised but its grammar and meaning are not analysed.
Misspelt drug name	Lexicon lookup	Surface form is absent from the lexicon.

Compare clean and noisy samples using the same labelled metrics: entity precision and recall, analyser coverage and retrieval F1. Weighted automata or edit-distance matching can handle near-spellings; character n-gram language identification can help identify language at token level. These methods should be evaluated for false corrections, especially for patient identifiers and drug names.

C. ADAPTATION OR REDESIGN (2 MARKS)

The recommended decision is MODIFY rather than replace the finite-state layer. The existing exact rules remain valuable for clean and safety-critical formats, but they should be supplemented by a controlled noise-handling front end.

C.1 Revised pipeline

OCR / typed input → line-break repair → character-noise candidate generation → tokenisation → token-level language identification → exact regex and DFA checks → weighted fuzzy matching for selected fields → English / regional lexicons and FSTs → search index

- Patterns: retain strict patterns for identifiers, dates and dosage values; add separately validated alternatives for common OCR confusions.
- Automata: retain exact DFA validation; use a weighted or edit-distance candidate stage before selected validators. Never silently rewrite high-risk identifiers.
- Lexicon: add frequent regional-language stems, common transliterations, approved abbreviations and misspelling variants with source and confidence labels.
- Rule cascade: add line-break repair and language-specific normalisation before morphological analysis.
- Fallback: use Porter stemming only for unrecognised English-like tokens, not for every token indiscriminately.

D. EVIDENCE-BASED DEFENCE (2 MARKS)

Modification is preferable because the original finite-state components remain accurate and efficient for clean, structured input. Replacing them entirely would discard useful exact validation. A weighted correction stage improves tolerance of OCR variation, while language identification and expanded lexicons address code-mixing. The design should expose uncertain candidates for review instead of silently choosing one.

The final defence should quote measured results from the student's clean and noisy samples: changes in precision, recall, coverage, F1, runtime and annotation effort. Until those experiments are run, the direction of the expected effect can be explained, but numerical improvement must not be claimed.

Remaining limitations

Even an extended finite-state layer cannot reliably infer every intended meaning from context, understand arbitrary clinical reasoning, resolve all ambiguous abbreviations, or guarantee correctness for unseen languages and new terminology. Human review and context-sensitive language models may still be needed for uncertain cases. The normalisation layer must never replace clinician judgement.

CONCLUSION

This assignment demonstrates how regular expressions, finite-state automata, morphological rules, finite-state transducers and stemming can be combined to improve hospital-record search. Regular expressions identify structured strings and support tokenisation. Automata provide precise validation. Inflectional and derivational morphology explain how clinical words are formed. An FST connects surface forms with lexical stems and grammatical features, while the Porter stemmer offers a lightweight baseline.

The recommended solution is a hybrid pipeline that preserves original text, uses exact rules where formats are predictable, applies morphological analysis where lexical resources are available, and adds carefully controlled noise handling for OCR and code-mixed input. Evaluation must be based on a documented corpus and manually judged queries. Accuracy, recall, runtime and maintenance effort should all influence the final design.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**